

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 30%

Dr.G.Ayyappan

Schema Used in All Questions:

Student (RollNo, Name, DeptID, Age, Marks) | Department (DeptID, DeptName, HOD)

Sample Data (load this into MySQL before attempting the questions):

Department

DeptID	DeptName	HOD
1	Computer Science	Dr. Rao
2	Electronics	Dr. Iyer
3	Mechanical	Dr. Nair

Student

RollNo	Name	DeptID	Age	Marks
101	Aravind	1	20	85
102	Bhavya	1	21	78
103	Chandran	1	20	90
104	Deepak	2	22	60
105	Esha	2	21	55
106	Farhan	2	22	95
107	Gowri	3	20	40
108	Hari	3	21	45
109	Indra	3	20	68

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Identify and fix the error in the given SQL query that uses incorrect JOIN syntax and returns duplicate results. Given Query: <pre>SELECT s.Name, d.DeptName FROM Student s, Department d; -- Intent: list each student with their own department's name</pre>	BL3 – Apply	5
2	The following sub-query returns incorrect results due to a missing correlation. Debug and rewrite it correctly. Given Query: <pre>SELECT s.Name, s.DeptID, s.Marks FROM Student s WHERE s.Marks > (SELECT AVG(Marks) FROM Student); -- Intent: list students who score above their OWN department's average</pre>	BL3 – Apply	5
3	A given SQL view definition causes errors when queried. Identify the issue and rewrite a correct and optimized view. Given Query: <pre>CREATE VIEW DeptAvgMarks AS SELECT Name, DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY DeptID; SELECT * FROM DeptAvgMarks;</pre>	BL3 – Apply	5
4	Debug the given relational algebra expression that produces incorrect output for a natural join operation. Given Query: <pre>$\pi(\text{Name, Age})(\text{Student}) \bowtie \text{Department}$ -- Intent: natural join of Student and Department on DeptID</pre>	BL3 – Apply	5
5	The SQL query below violates referential integrity. Identify the violation and modify the statements to enforce proper constraints. Given Query: <pre>CREATE TABLE Student (RollNo INT PRIMARY KEY, Name VARCHAR(50), DeptID INT, Age INT, Marks INT); -- Department table only has DeptID values 1, 2, 3 INSERT INTO Student (RollNo, Name, DeptID, Age, Marks) VALUES (110, 'Arun', 9, 20, 78);</pre>	BL3 – Apply	5

6	Given an inefficient SQL query with nested loops, rewrite it using JOIN to optimize performance. Given Query: <pre>SELECT s.Name, (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID) AS DeptName FROM Student s;</pre>	BL6 – Create	5
7	The GROUP BY query below produces wrong aggregation results. Debug the query and explain the error. Given Query: <pre>SELECT DeptID, AVG(Marks) AS AvgMarks FROM Student GROUP BY RollNo;</pre>	BL3 – Apply	5
8	Identify and fix the logical error in a given SQL UPDATE statement that unintentionally modifies all rows in the table. Given Query: <pre>-- Intent: give 5 bonus marks to students in Department 2 only UPDATE Student SET Marks = Marks + 5;</pre>	BL3 – Apply	5
9	Optimize the given multi-table SQL query by restructuring sub-queries into JOINS and using appropriate indexes. Given Query: <pre>SELECT s.Name, s.Marks, (SELECT DeptName FROM Department WHERE DeptID = s.DeptID) AS DeptName, (SELECT HOD FROM Department WHERE DeptID = s.DeptID) AS HeadOfDept FROM Student s WHERE s.DeptID IN (SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics'));</pre>	BL6 – Create	5
10	Debug a given relational calculus expression that returns an empty result set due to incorrect quantifier usage. Given Query: <pre>-- Intent: students who scored more than AT LEAST ONE student in Department 2 { s.Name Student(s) ∧ (∀ t) (Student(t) ∧ t.DeptID = 2 → s.Marks > t.Marks) }</pre>	BL3 – Apply	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Question 1:

Incorrect JOIN causing duplicate results

Given Wrong Query:

```
SELECT s.Name, d.DeptName  
FROM Student s, Department d;
```

- This is a CARTESIAN PRODUCT, not a join.
- It returns every student paired with every department → duplicates.

Correct Query:

```
SELECT s.Name, d.DeptName  
FROM Student s  
JOIN Department d ON s.DeptID = d.DeptID;
```

Question 2:

Incorrect correlated subquery

Given Wrong Query:

```
SELECT s.Name, s.DeptID, s.Marks  
FROM Student s  
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
```

- Compares marks with **overall average**, not department average.
- Missing correlation on DeptID.

Correct Correlated Subquery:

```
SELECT s.Name, s.DeptID, s.Marks  
FROM Student s  
WHERE s.Marks > (  
    SELECT AVG(Marks)  
    FROM Student  
    WHERE DeptID = s.DeptID  
);
```

Question 3:

Incorrect SQL View

Given Wrong View:

```
CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

- Name is **not** in GROUP BY → invalid.
- View mixes aggregated and non-aggregated columns.

Correct Optimized View:

```
CREATE VIEW DeptAvgMarks AS
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

Question 4:

Incorrect Relational Algebra Natural Join

Given Wrong Expression:

$$\pi(\text{Name, Age})(\text{Student}) \bowtie \text{Department}$$

Correct Expression:

$$\pi(\text{Name, Age})(\text{Student} \bowtie \text{Student.DeptID} = \text{Department.DeptID} \text{Department})$$

Question 5:

Referential Integrity Violation

Given Wrong Insert:

```
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 9, 20, 78);
```

- DeptID = 9 does **not exist** in Department table (only 1,2,3).
- Violates FOREIGN KEY constraint.

Correct Insert:

```
INSERT INTO Student (RollNo, Name, DeptID, Age, Marks)
VALUES (110, 'Arun', 2, 20, 78);
```

Or enforce FK explicitly:

```
ALTER TABLE Student
ADD FOREIGN KEY (DeptID) REFERENCES Department(DeptID);
```

Question 6:

Inefficient Nested Subquery

Given Inefficient Query:

```
SELECT s.Name,
       (SELECT d.DeptName FROM Department d WHERE d.DeptID = s.DeptID)
FROM Student s;
```

- Subquery runs for **every row** → slow.

Optimized JOIN:

```
SELECT s.Name, d.DeptName
FROM Student s
JOIN Department d ON s.DeptID = d.DeptID;
```

Question 7:

Wrong GROUP BY

Given Wrong Query:

```
SELECT DeptID, AVG(Marks)
FROM Student
GROUP BY RollNo;
```

- Grouping by RollNo produces **one row per student**, not per department.
- DeptID is not grouped → incorrect aggregation.

Correct Query:

```
SELECT DeptID, AVG(Marks) AS AvgMarks
FROM Student
```

GROUP BY DeptID;

Question 8:

Incorrect UPDATE modifying all rows

Given Wrong Query:

```
UPDATE Student
SET Marks = Marks + 5;
```

- No WHERE clause → updates **every student**.

Correct Query:

```
UPDATE Student
SET Marks = Marks + 5
WHERE DeptID = 2;
```

Question 9:

Optimize Multi-Table Query

Given Inefficient Query:

Multiple subqueries:

```
SELECT s.Name, s.Marks,
       (SELECT DeptName FROM Department WHERE DeptID = s.DeptID),
       (SELECT HOD FROM Department WHERE DeptID = s.DeptID)
FROM Student s
WHERE s.DeptID IN (
    SELECT DeptID FROM Department WHERE DeptName IN ('Computer Science', 'Electronics')
);
```

Optimized JOIN Version:

```
SELECT s.Name, s.Marks, d.DeptName, d.HOD
FROM Student s
JOIN Department d ON s.DeptID = d.DeptID
WHERE d.DeptName IN ('Computer Science', 'Electronics');
```

Question 10:

Incorrect Relational Calculus Quantifier

Given Wrong TRC:

$$\{ s.\text{Name} \mid \text{Student}(s) \wedge (\forall t)(\text{Student}(t) \wedge t.\text{DeptID} = 2 \rightarrow s.\text{Marks} > t.\text{Marks}) \}$$

- Uses $\forall t \rightarrow$ requires $s.\text{Marks} > \text{ALL}$ students in Dept 2.
- Intent: more than **at least one** student.

Correct TRC ($\exists t$):

$$\{ s.\text{Name} \mid \text{Student}(s) \wedge \exists t(\text{Student}(t) \wedge t.\text{DeptID} = 2 \wedge s.\text{Marks} > t.\text{Marks}) \}$$
